Deleting variable templates

Document Number: P2041R0
Date: 2020-01-11
Author: David Stone (david.stone@uber.com, david@doublewise.net)
Audience: Evolution Working Group (EWG)

Motivating example

```
template<typename T>
auto max_value = delete;

template<std::unsigned_integral T>
constexpr auto max_value<T> = T(-1);

template<>
constexpr std::int8_t max_value<std::int8_t> = 127;

template<>
constexpr std::int8_t max_value<std::int16_t> = 32767;

template<>
constexpr std::int8_t max_value<std::int32_t> = 2147483647;

template<>
constexpr std::int8_t max_value<std::int64_t> = 9223372036854775807;

template<typename T>
concept maxable = requires { max_value<T>; };

static_assert(maxable<int>);
static_assert(!maxable<std::string>);
```

Recommended solution

Variable templates are the natural way to define a value trait: it is explicitly a value templated on some type. The alternative is the more old-school approach of defining a trait in terms of a class template with a `static constexpr auto value` member, and then define a variable template that removes the boilerplate. This ends up duplicating your API and leads to natural questions of which version a user wants for reading the value vs. specializing the value. A SFINAE-friendly approach to removing the primary template of a variable template is the last missing piece to

making variable templates a direct expression of users' intent, and using the syntax `= delete` is the natural way to express this intent.

Attempt to do this today: constraints

A first attempt to do this today might look like this:

```
template<typename T> requires false
int max_value;

template<std::unsigned_integral T>
constexpr auto max_value<T> = T(-1);

template<>
constexpr std::int8_t max_value<std::int8_t> = 127;
```

This immediately runs into a few compiler errors:

```
<source>:14:15: error: type constraint differs in template redeclarati

template<std::unsigned_integral T>

              ^

<source>:11:10: note: previous template declaration is here

template<typename T> requires false

         ^

<source>:18:23: error: constraints not satisfied for variable template

constexpr std::int8_t max_value<std::int8_t> = 127;

                      ^~~~~~~~~~~~~~~~~~~~~

<source>:11:31: note: because 'false' evaluated to false

template<typename T> requires false

                               ^

2 errors generated.

Compiler returned: 1
```

Attempt to do this today: always invalid expression

```
template<typename T>
auto max_value = not defined(max_value<T>);
// Silly code credit to Richard Smith; note that this is equivalent to
// auto max_value = sdghdfksljadslfkjdslfk<T>();

template<std::unsigned_integral T>
constexpr auto max_value<T> = T(-1);

template<>
constexpr std::int8_t max_value<std::int8_t> = 127;

template<typename T>
concept maxable = requires { max_value<T>; };

static_assert(!maxable<void>);
```

Rather than evaluating to `false`, the `maxable<void>` expression is a hard error.

```
<source>:12:22: error: use of undeclared identifier 'defined'

auto max_value = not defined(max_value<T>);

                     ^

<source>:23:30: note: in instantiation of variable template specializa

concept maxable = requires { max_value<T>; };

                             ^

<source>:23:30: note: in instantiation of requirement here

concept maxable = requires { max_value<T>; };

                             ^~~~~~~~~~~~

<source>:23:19: note: while substituting template arguments into const

concept maxable = requires { max_value<T>; };

                  ^~~~~~~~~~~~~~~~~~~~~~~~~

<source>:25:16: note: while checking the satisfaction of concept 'maxa

static_assert(!maxable<void>);
```

```
                ^~~~~~~~~~~~~

1 error generated.

Compiler returned: 1
```

Attempt to do this today: extern incomplete

```
struct incomplete;

template<typename>
extern incomplete max_value;

template<std::unsigned_integral T>
constexpr auto max_value<T> = T(-1);

template<>
constexpr std::int8_t max_value<std::int8_t> = 127;

template<typename T>
concept maxable = !std::is_same_v<decltype(max_value<T>), incomplete>;

static_assert(!maxable<void>);
```

This all works correctly. Unfortunately, this, too, has drawbacks. Users of this variable template need to know that they cannot just rely on normal substitution failures and thus they cannot use it directly in a requires clause as part of a larger test for expression validity. Instead, all users need to do a test with `is_same_v`. Moreover, there does not appear to be a way to prevent users from passing around a reference or pointer to an instantiation of the primary template. This appears to be the best we can do under current rules.

Chosen syntax

We can delete functions, including the primary template of a function, with the behavior needed for variable templates. The following code is valid today:

```
template<typename T>
void foo() = delete;

template<std::unsigned_integral T>
void foo() {
}

template<>
```

```
void foo<int>() {
}

template<typename T>
concept fooable = requires { foo<T>(); };

static_assert(fooable<unsigned>);
static_assert(fooable<int>);
static_assert(!fooable<void>);
```

The `= delete` syntax has become an intuitive way for users to say that some portion of an otherwise generated interface should not actually be considered valid code, and that using a deleted overloads or specializations should be a substitution failure and interact usefully with SFINAE. Deleting a variable template (either the primary template or an explicit specialization) is a natural evolution.

Another possible concern is that this syntax could possibly be ambiguous with operator delete, but this does not appear to be the case. Existing code that would look most like this falls into one of two forms:

```
template<typename T>
T thing = operator delete;
// Requires that T is something like void(*)(void *)
```

and

```
template<typename T>
T thing = (delete ptr, T());
```

In other words, using `delete` as a pointer to function requires the `operator` keyword before it, and using `delete` as an expression requires an operand after it.

What about class templates?

Once we can delete a variable template in addition to our current ability to delete functions and function templates, the next obvious extension is to be able to delete a class template. This would be similar to declaring but not defining the class ("defining" it as incomplete) except that users could not even form a pointer or reference to such a type. I am aware of no major use cases for this feature, and as such, I am not proposing it at this time.